

ECE 113: Digital Signal Processing

Final Project

Deadline: Sunday 11:55 PM, June 07, 2026

Overall Project Description

This final project consists of two main parts designed to test your understanding and application of fundamental Digital Signal Processing (DSP) concepts:

- **Part 1: From Basic Time-Frequency Analysis to Multi-User Communications.** You will build a communication system from the ground up, utilizing your knowledge of the Discrete-Time Fourier Transform (DTFT), Linear Time-Invariant (LTI) systems, and orthogonality to discover how modern multi-user communication networks efficiently share spectrum.
 - **Part 2: Audio Filtering Using FIR and IIR Discrete-Time Filters.** You will study how discrete-time systems affect an audio signal. You will add controlled artificial noise to a clean speech signal and analyze the effects of given FIR and IIR filters in both the time and frequency domains.
-

Part 1: From Basic Time-Frequency Analysis to Multi-User Communications

The Multi-User Uplink Problem

Scenario: User 1 wishes to transmit a discrete-time signal $x_1[n]$ at a carrier frequency f_1 , and User 2 wishes to transmit a signal $x_2[n]$ at a carrier frequency f_2 to the same Base Station (BS). Both users transmit a rectangular pulse of length N samples, with a sampling interval of T_s . The baseband transmitted signals carry a data symbol of $+1$ or -1 . The antenna at the Base Station receives the combined discrete-time signal:

$$y[n] = x_1[n] + x_2[n], \quad \text{for } n = 0, 1, \dots, N-1 \quad (1)$$

Task 1.1: Traditional FDM, LPF, and the MSE Metric

Assume the system operates with a sampling frequency $f_s = 1000$ Hz ($T_s = 0.001$ s) and processes data in blocks of $N = 100$ samples. The total block duration is $T = NT_s = 0.1$ seconds. Let User 1's carrier frequency be $f_1 = 100$ Hz, and the frequency spacing between the two users be $\Delta f = f_2 - f_1$. To transmit their respective data symbols $d_1 = 1$ and $d_2 = -1$,

each user modulates a single complex exponential carrier. Over a block of N samples, the transmitted signals are:

$$x_1[n] = d_1 e^{j2\pi f_1 n T_s}, \quad \text{for } n = 0, 1, \dots, N-1 \quad (2)$$

$$x_2[n] = d_2 e^{j2\pi f_2 n T_s}, \quad \text{for } n = 0, 1, \dots, N-1 \quad (3)$$

The received signal is $y[n] = x_1[n] + x_2[n]$. To decode User 1's message, the receiver first shifts the spectrum of the received signal so that User 1's carrier is centered at baseband (BB, 0 Hz). By multiplying $y[n]$ with the conjugate carrier $e^{-j2\pi f_1 n T_s}$, we obtain the down-converted signal:

$$y_{BB}[n] = y[n] e^{-j2\pi f_1 n T_s} = d_1 + d_2 e^{j2\pi \Delta f n T_s} \quad (4)$$

To systematically evaluate the quality of the recovered signal $r[n]$, we will use the Mean Squared Error (MSE) compared to the ideal transmitted symbol d_1 :

$$\text{MSE} = \frac{1}{N} \sum_{n=0}^{N-1} (\text{Re}\{r[n]\} - d_1)^2 \quad (5)$$

- Direct Decoding (No Filter):** Implement the signals in MATLAB. Assume a naive receiver that directly uses the down-converted signal without filtering ($r[n] = y_{BB}[n]$). Test with the specific frequency spacings $\Delta f \in \{5, 10, 40, 100\}$ Hz and compute the MSE for each case. Observe how the second term heavily interferes with your decision.
- LPF Receiver:** Now, apply a standard FIR Low Pass Filter (LPF) to $y_{BB}[n]$ before decoding. Use MATLAB's built-in function to design a 30th-order filter with a normalized cutoff frequency of 0.1 (which corresponds to 50 Hz): `lpf = fir1(30, 0.1)`. Apply it using `r = filtfilt(lpf, 1, y_BB)` to ensure zero-phase distortion. Test with the same spacing values $\Delta f \in \{5, 10, 40, 100\}$ Hz, compute the MSE for each, and plot the filtered time-domain signal $r[n]$ for the $\Delta f = 10$ Hz and 100 Hz cases.
- Guard Band and spacing:** Based on your MSE results, determine the minimum spacing (Guard Band) required for the LPF to cleanly separate the signals ($\text{MSE} \approx 0$). Discuss how this requirement affects the overall spectrum efficiency of the system. Then, specifically examine the case where $\Delta f = 10$ Hz. Note that this spacing corresponds exactly to $\Delta f = \frac{1}{NT_s} = \frac{1}{T}$. Explain why the LPF completely fails to separate the signals at this specific tight spacing, motivating the need for a completely different receiver design in the next task.

Task 1.2: A New Receiver and the Critical Spacing

Let us abandon the LPF and formally apply the concept of inner products (correlation) in the discrete-time domain. To decode User 1's symbol, the Base Station correlates the received block $y[n]$ with User 1's carrier signal. This involves multiplying $y[n]$ by the conjugate carrier $e^{-j2\pi f_1 n T_s}$ and summing the result over all N samples. Since this point-by-point multiplication exactly yields our previously defined down-converted signal $y_{BB}[n]$, the normalized decoded symbol $\hat{d}_1$ (which is generally complex-valued) for User 1 can be explicitly written as:

$$\hat{d}_1 = \frac{1}{N} \sum_{n=0}^{N-1} y[n] e^{-j2\pi f_1 n T_s} = \frac{1}{N} \sum_{n=0}^{N-1} y_{BB}[n] \quad (6)$$

- The Correlation Receiver:** Implement this correlation-based receiver in MATLAB. Treat Δf as a continuous sweepable variable from 0 to 100 Hz to match your LPF experiments (use a step size of 0.5 Hz or smaller). For each value of Δf , obtain the complex-valued $\hat{d}_1$ and calculate the squared magnitude error of your decision on the complex plane: $E = |\hat{d}_1 - d_1|^2$. Plot this error E versus Δf .

- b. **Observation & Discovery:** Based on your plot, describe the behavior of the error curve. Specifically, what happens to the error at certain frequency spacings, even though their frequency spectra heavily overlap? From your empirical results, identify the minimum non-zero spacing Δf (in terms of N and T_s) that yields exactly zero error.
- c. **The Mathematical Proof of Orthogonality:** As engineers, we must prove that the empirical observation in (b) is not a coincidence. Write down the discrete-time summation for the interference term caused solely by User 2 at User 1's receiver:

$$I = \frac{1}{N} \sum_{n=0}^{N-1} e^{j2\pi f_2 n T_s} e^{-j2\pi f_1 n T_s} = \frac{1}{N} \sum_{n=0}^{N-1} e^{j2\pi \Delta f n T_s} \quad (7)$$

Using the formula for the sum of a finite geometric series, mathematically prove that when the frequency spacing is exactly the value you discovered in (b) (i.e., $\Delta f = \frac{k}{NT_s}$ where k is a non-zero integer), the interference I is exactly zero. This foundational concept is known as **Discrete Orthogonality**.

Task 1.3: Scaling Up to N Users

The Base Station is now serving a massive number of users, N , transmitting simultaneously on the uplink. To completely avoid interference, the Base Station mandates that each user k must transmit their data symbol $d_k \in \{+1, -1\}$ using a unique carrier frequency perfectly aligned with the critical spacing you discovered in Task 1.2:

$$f_k = \frac{k}{NT_s}, \quad \text{for } k = 0, 1, \dots, N-1 \quad (8)$$

The Base Station antenna receives the superposition (sum) of all N independent signals simultaneously over a block of N samples. Let this combined received discrete-time signal be $y[n]$.

- a. **Receive signal model:** Write down the mathematical expression for the combined signal $y[n]$ received at the Base Station antenna. Carefully observe the structure of this summation. Does it look familiar? Compare your equation with the fundamental DSP transforms you learned in class (Hint: Think about DTFS, DTFT, DFT, or IDFT). What well-known transform does it apply?
- b. **Decoding in Receiver:** In Task 1.2, you proved that the correlation receiver perfectly recovers a symbol without interference due to discrete orthogonality. Now, generalize this. Write down the mathematical expression for the Base Station receiver to calculate the estimated symbol $\hat{d}_k$ for any arbitrary user k from the combined signal $y[n]$. Similar to (a), what standard DSP transform exactly represents this massive multi-user demodulation process?
- c. **The Multi-User Simulation:** Assume $N = 1024$. Generate a random sequence of $+1$ and -1 for the N data symbols (d_k). Using your algorithmic discoveries from (a) and (b), write a MATLAB script to simulate the uplink transmission and the Base Station receiver using built-in fast transform functions. Assume an ideal channel. Calculate the average Squared Magnitude Error across all N users: $\text{MSE}_{avg} = \frac{1}{N} \sum_{k=0}^{N-1} |\hat{d}_k - d_k|^2$.

Note: You have just independently derived and implemented the core modulation and demodulation blocks of Orthogonal Frequency Division Multiple Access (OFDMA)!

Part 2: Audio Filtering Using FIR and IIR Discrete-Time Filters

In this project, you will study how discrete-time systems affect an audio signal. You will start with a clean speech signal, add controlled artificial noise components, and then apply two given filters: one FIR filter and one IIR filter. The goal is to understand how different filters affect the signal in the time domain, frequency domain, and by listening to the audio before and after filtering. This project does not ask you to design filters. The filters are already given to you in terms of their system functions, $H(z)$. Your task is to analyze their frequency responses, apply them to a noisy speech signal, and explain the results.

Provided Audio File

You are given a clean speech audio file which can be found under Bruinlearn week 7 module:

`speech_clean_16k.wav.`

This audio file has already been converted to mono and sampled at

$$F_s = 16 \text{ kHz}.$$

Therefore, the highest representable frequency is

$$\frac{F_s}{2} = 8 \text{ kHz}.$$

All students should use this same audio file so that everyone obtains the same results.

Task 2.1: Create and Listen to a Noisy Speech Signal

First, load the clean speech signal in Matlab. Then create a noisy version by adding three noise components:

1. Broadband white noise.
2. A sinusoidal tone at 3.2 kHz, close to the upper part of the speech band.
3. A sinusoidal tone at 5.5 kHz, farther away from the main speech band.

Although the fundamental pitch of speech is often around 80–250 Hz, the important intelligibility content of speech extends much higher, often up to about 3–4 kHz. Therefore, a tone at 3.2 kHz is close to the useful speech band, while a tone at 5.5 kHz is farther away and should be easier to suppress by a low-pass filter. Use the following Matlab code to load the clean signal and create the noisy signal.

```
1 clear; close all; clc;
2
3 % Load the provided clean speech signal
4 [x, Fs] = audioread('speech_clean_16k.wav');
5
6 % Convert to mono if needed
7 if size(x, 2) > 1
8     x = mean(x, 2);
9 end
10
11 % Check that the sampling frequency is 16 kHz
```

```

12 if Fs ~= 16000
13     error('The provided audio file should have Fs = 16 kHz.');
```

```

14 end
15
16 % Remove DC offset and normalize clean speech
17 x = x - mean(x);
18 x = x / max(abs(x));
19
20 % Time axis
21 n = (0:length(x)-1)';
22 t = n/Fs;
23
24 % Noise tone frequencies
25 f_close = 3200; % Hz, close to upper speech band
26 f_far    = 5500; % Hz, farther from speech band
27
28 % Relative noise amplitudes
29 % These values scale the noise relative to the RMS value of the clean
    speech.
30 white_scale = 0.15; % broadband white noise level
31 close_scale  = 0.40; % 3.2 kHz tone level
32 far_scale    = 0.40; % 5.5 kHz tone level
33
34 % Make the random white noise repeatable
35 rng(1);
36
37 % Compute RMS value of clean speech
38 speech_rms = sqrt(mean(x.^2));
39
40 % Set actual noise amplitudes relative to speech RMS
41 A_white = white_scale * speech_rms;
42 A_close  = close_scale * speech_rms;
43 A_far    = far_scale   * speech_rms;
44
45 % Generate noise components
46 white_noise = A_white * randn(size(x));
47 tone_close  = A_close * sin(2*pi*f_close*t);
48 tone_far    = A_far   * sin(2*pi*f_far*t);
49
50 % Add noise to clean speech
51 % This raw noisy signal is useful for spectrum analysis.
52 x_noisy_raw = x + white_noise + tone_close + tone_far;
53
54 % Normalize only for playback/export to avoid clipping
55 x_noisy = x_noisy_raw / max(abs(x_noisy_raw));
56
57 % Save noisy speech
58 audiowrite('speech_noisy_16k.wav', x_noisy, Fs);
59
60 % Listen to clean and noisy speech
61 disp('Playing clean speech...');
62 sound(x, Fs);
63 pause(length(x)/Fs + 1);
64
65 disp('Playing noisy speech...');
66 sound(x_noisy, Fs);

```

The noise amplitudes are selected relative to the RMS value of the clean speech signal. For

example,

$$A_{\text{white}} = \text{white_scale} \times \text{speech RMS}.$$

The white noise amplitude controls the strength of the broadband background noise. The two sinusoidal amplitudes control the strength of the narrowband interference tones. Since the sinusoidal tones are concentrated at single frequencies, they can be very noticeable even when their amplitudes are not very large. The command

$$\text{rng}(1)$$

makes the random white noise repeatable. This means that all students generate the same white noise sequence and obtain consistent plots and audio results.

1. Plot the clean and noisy speech signals in the time domain.

Task 2.2: Frequency-Domain Analysis of the Clean and Noisy Signals

Plot the magnitude spectra of both the clean speech signal and the noisy speech signal. Use frequency in Hz on the horizontal axis and plot from 0 to 8 kHz. In the noisy signal spectrum, identify:

1. The broadband white noise contribution.
2. The sinusoidal tone near 3.2 kHz.
3. The sinusoidal tone near 5.5 kHz.

In your report, comment on how the added noise components appear in the frequency domain.

Task 2.3: Analysis of the Given FIR and IIR Filters

You are given two discrete-time filters. Both filters are low-pass filters with approximately the same -3 dB cutoff frequency near

$$f_c \approx 2.7 \text{ kHz}$$

when the sampling frequency is

$$F_s = 16 \text{ kHz}.$$

Therefore, both filters approximately pass frequencies below 2.7 kHz and attenuate frequencies above that range. However, they have different roll-off behavior and different phase behavior.

The first filter is a 4th-order FIR filter:

$$H_{\text{FIR}}(z) = 0.0177 + 0.2287z^{-1} + 0.5072z^{-2} + 0.2287z^{-3} + 0.0177z^{-4}.$$

The corresponding difference equation is

$$\begin{aligned} y_{\text{FIR}}[n] = & 0.0177x[n] + 0.2287x[n-1] + 0.5072x[n-2] \\ & + 0.2287x[n-3] + 0.0177x[n-4]. \end{aligned}$$

The second filter is a 4th-order IIR filter:

$$H_{\text{IIR}}(z) = \frac{0.0532 + 0.0304z^{-1} + 0.0845z^{-2} + 0.0304z^{-3} + 0.0532z^{-4}}{1 - 2.1045z^{-1} + 2.4080z^{-2} - 1.4106z^{-3} + 0.3896z^{-4}}.$$

The corresponding difference equation is

$$\begin{aligned} y_{\text{IIR}}[n] = & 2.1045y_{\text{IIR}}[n-1] - 2.4080y_{\text{IIR}}[n-2] \\ & + 1.4106y_{\text{IIR}}[n-3] - 0.3896y_{\text{IIR}}[n-4] \\ & + 0.0532x[n] + 0.0304x[n-1] + 0.0845x[n-2] \\ & + 0.0304x[n-3] + 0.0532x[n-4]. \end{aligned}$$

For each filter:

1. Plot the magnitude response in dB.
2. Plot the phase response.
3. Comment on whether the phase response is approximately linear or nonlinear.
4. Compare how much each filter attenuates frequencies near 3.2 kHz and 5.5 kHz.
5. Plot the time domain impulse response of each filter. For the IIR filter, plot only the first 100 samples.

Task 2.4: Apply the FIR and IIR Filters to the Noisy Signal

Apply both filters to the noisy speech signal. Then plot the magnitude spectra of:

1. The noisy speech signal.
2. The FIR-filtered speech signal.
3. The IIR-filtered speech signal.

Compare how much each filter suppresses the sinusoidal tones near 3.2 kHz and 5.5 kHz. In your report, answer the following questions:

1. Which filter removes the high-frequency tones more effectively?
2. Which filter has a sharper transition from low frequencies to high frequencies?
3. Which filter has a more linear phase response?
4. What tradeoff do you observe between filtering strength and phase behavior?

Task 2.5: Listen to the Filtered Audio

Save the FIR-filtered and IIR-filtered signals as audio files:

`speech_filtered_FIR.wav`

`speech_filtered_IIR.wav`

Listen to the clean, noisy, FIR-filtered, and IIR-filtered signals. In your report, describe what you hear. In particular, comment on:

1. Whether the annoying tones are reduced.
2. Whether the broadband noise is reduced.
3. Whether the speech sounds more natural after FIR or IIR filtering.
4. Whether the IIR output sounds cleaner, duller, sharper, or slightly distorted compared to the FIR output.

Submission Requirements

On Gradescope, there are two separate submission portals for this project:

- **Report Submission:** Upload your project report here as a single PDF.
- **Appended File Submission:** Compress all of the following files into a single ZIP archive and submit it here.
 1. Your Matlab code as one or multiple `.m` files.
 2. The generated noisy speech audio file: `speech_noisy_16k.wav`
 3. The FIR-filtered audio file: `speech_filtered_FIR.wav`
 4. The IIR-filtered audio file: `speech_filtered_IIR.wav`

For Part 1, your report should include:

1. A summary of the calculated MSE results for both the Direct and LPF receivers across the tested frequency spacings (Task 1.1).
2. Time-domain plots of the LPF recovered signal $r[n]$ for the $\Delta f = 10$ Hz and 100 Hz cases (Task 1.1).
3. A plot of the Squared Magnitude Error versus frequency spacing Δf for the correlation receiver (Task 1.2).
4. The complete mathematical proof demonstrating discrete orthogonality using the finite geometric series (Task 1.2).
5. The mathematical expressions for the multi-user received signal and the decoding process, explicitly identifying the corresponding DSP transforms (Task 1.3).
6. The average MSE results (Task 1.3)
7. Explanations that are asked to provide in Task 1.1, 1.2, and 1.3.

For Part 2, your report should include:

1. Time-domain plots of the clean and noisy signals.
2. Frequency-domain plots of the clean and noisy signals.
3. Magnitude and phase response plots for both filters.
4. Frequency-domain plots of the noisy, FIR-filtered, and IIR-filtered signals.
5. A short written comparison of the FIR and IIR filter responses.
6. A short listening-based discussion of the audio quality before and after filtering.

Please make sure your Matlab code can be run directly from the same folder containing the provided clean audio file, `speech_clean_16k.wav`. Your code should generate the noisy audio and the two filtered audio files listed above.